

# 제품 요구사항 문서 (PRD)

## 풀스택 메모장 (Full-Stack Memo App)

문서 버전: v1.0.0

작성일: 2025년 6월

작성자: PM / 풀스택 개발자

상태: 구현 완료 (MVP 기준)

### 1. 문서 개요

#### 1.1 프로젝트 소개

풀스택 메모장(Full-Stack Memo App)은 사용자가 간편하게 메모를 생성, 조회, 수정, 삭제할 수 있는 웹 기반 메모 관리 애플리케이션입니다. 본 프로젝트는 단순한 CRUD 기능을 넘어, 실무 수준의 프론트엔드 아키텍처 설계와 백엔드 RESTful API 연동을 성공적으로 결합하고 프로덕션 급의 웹 성능 최적화를 검증하기 위한 목적으로 개발되었습니다.

#### 1.2 개발 목적

이 프로젝트의 주요 목적은 다음과 같습니다.

- 풀스택 개발 역량 시연:** 프론트엔드(React)와 백엔드(Node.js/Express)를 직접 연결하는 실질적인 클라이언트-서버 구조를 구현합니다.
- React 심화 학습:** `useState`, `useEffect`, `useRef`, `useCallback`, `useMemo`, `React.memo` 등 핵심 내장 Hook을 명확한 아키텍처 원칙하에 조합하고 적용합니다.
- UX 완성도 제고:** 애니메이션 효과, 실시간 검색 하이라이팅, 특수 키보드 인터랙션 등 사용자 중심의 디테일한 편의 기법을 제공합니다.
- 엔지니어링 코드 품질 확보:** 명확한 컴포넌트 단위 분리, 메모이제이션을 통한 가상 DOM 연산 비용 절감 등 프로덕션 환경에 준하는 성능 가이드를 수립합니다.

1.3 핵심 가치 (Core Values)

| 가치 구분                   | 상세 내용 설명                                               |
|-------------------------|--------------------------------------------------------|
| 단순함 (Simplicity)        | 인증 절차를 과감히 생략하고 진입 시 즉시 메모를 기입할 수 있는 직관적인 단일 창 레이아웃 구축 |
| 반응성 (Responsiveness)    | 클라이언트 상태와 백엔드 데이터베이스 간의 완전 동기화 및 실시간 스타일 동적 반영         |
| 효율성 (Efficiency)        | 복합적인 실시간 필터 및 검색 알고리즘으로 대량의 레코드 중 목적 데이터로의 초고속 접근성 확보  |
| 유지보수성 (Maintainability) | 하위 컴포넌트 분리를 통한 단일 책임 원칙 준수 및 향후 기능 결합 편의성 증대           |

2. 기술 스택 및 개발 환경

2.1 프론트엔드 (Frontend)

| 기술 구분   | 선택 핵심 기술         | 선택 배경 및 도입 효과                          |
|---------|------------------|----------------------------------------|
| 프레임워크   | React 18         | 선언적 상태 관리 기반 및 컴포넌트 지향 아키텍처 확립         |
| 번들러     | Vite             | Esbuilt 엔진을 활용한 초고속 HMR 및 경량 빌드 인프라 활용 |
| 스타일링    | Tailwind CSS     | 유틸리티 클래스를 적용한 신속한 인라인 스타일 배포 및 일관성 보장  |
| 상태 관리   | React Hooks (내장) | 외부 미들웨어 오버헤드 배제로 단일 진입 파일 내 완결성 유도     |
| HTTP 통신 | Fetch API (내장)   | 추가 번들 사이즈 증가 없이 브라우저 네이티브 비동기 표준 스펙 활용 |

구현 방식 사양: `App.jsx` 단일 파일 구조로 전체 핵심 상태 비즈니스 로직을 중앙 집약적으로 통제하되, 단위 요소 표현부인 `MemoCard` 를 하위 독립 컴포넌트로 구조화하여 관심사 분리(Separation of Concerns)를 구현했습니다.

## 2.2 백엔드 (Backend)

| 아키텍처 레이어 | 적용 기술 명세               | 비고 및 세부 환경 설정                                       |
|----------|------------------------|-----------------------------------------------------|
| 런타임 환경   | Node.js                | 비동기 이벤트 기반 대규모 I/O 처리 및 JS 단일 언어 스택 형성              |
| 웹 프레임워크  | Express.js             | 경량 미들웨어 기반 라우팅 제어 및 RESTful 엔드포인트 설계                |
| 데이터 아카이빙 | 인메모리 (Memory Store)    | 서버 프로세스 내부의 <code>memos</code> 자바스크립트 전역 배열 오브젝트 활용 |
| 진입 파일    | <code>server.js</code> | 단일 인프라 구성 파일로 통합 라우트 미들웨어 통제                        |
| 네트워크 포트  | <code>5000</code>      | 프론트엔드 내부 Vite 개발 포트(5173)와 충돌을 배제한 지정 바인딩           |
| 자원 보안 정책 | <code>cors</code> 미들웨어 | 로컬 크로스 오리진 상황에서의 통신 자원 접근 규약 완화 정책 적용               |

## 2.3 API 명세 (RESTful API Specification)

백엔드는 아래 명시된 4개의 명확한 규격화 엔드포인트를 제공하며 통신 프로토콜 규약을 철저히 준수합니다.

Base URL: `http://localhost:5000`

| 메서드    | 엔드포인트                   | 기능 명세             | 요청 Body 스키마                     | 반환 데이터 규격                      |
|--------|-------------------------|-------------------|---------------------------------|--------------------------------|
| GET    | <code>/todos</code>     | 전체 메모 데이터 스냅샷 조회  | 없음                              | <code>Array&lt;Memo&gt;</code> |
| POST   | <code>/todos</code>     | 새 메모 레코드 영석 적재 요청 | <code>{ title, content }</code> | 생성 완료된 <code>Memo</code>       |
| PATCH  | <code>/todos/:id</code> | 메모 중요도 플래그 동적 반전  | <code>{ important }</code>      | 수정 완료된 <code>Memo</code>       |
| DELETE | <code>/todos/:id</code> | 메모 물리 ID 타킷 제거    | 없음                              | <code>{ success: true }</code> |

## 2.4 데이터 모델 (Data Schema)

서버 메모리 버퍼 레이어에서 규격화되어 처리되는 인터페이스 데이터 스키마는 다음과 같습니다.

```
{
  "id": "1717000000000",
  "title": "주간 업무 보고서 작성",
  "content": "- 프론트엔드 최적화 작업 검증",
  "- 신규 REST API 연동 완료",
  "important": false,
  "createdAt": "2025-06-01 14:30"
}
```

| 필드명                    | 데이터 타입               | 필수 여부   | 필드 상세 설명 및 제약 조건                                        |
|------------------------|----------------------|---------|---------------------------------------------------------|
| <code>id</code>        | <code>String</code>  | 시스템 생성  | <code>Date.now()</code> 기반 고유 유닉스 밀리초 문자열 식별자           |
| <code>title</code>     | <code>String</code>  | ✓ 필수 항목 | 메모 대제목 (트림 처리 기준 공백 입력 전면 금지)                           |
| <code>content</code>   | <code>String</code>  | ✓ 필수 항목 | 메모 본문 데이터 레코드 (멀티 라인 개행 문자열 보존)                         |
| <code>important</code> | <code>Boolean</code> | ✓ 필수 항목 | 중요 항목 분류용 논리 지표 (초기 적재 디폴트: <code>false</code> )        |
| <code>createdAt</code> | <code>String</code>  | 시스템 생성  | 현지 호스트 시간대 기준 <code>"YYYY-MM-DD HH:mm"</code> 포맷 상수 포매팅 |

## 3. 기능적 요구사항

### 3.1 메모 생성 (Create)

- **UI 및 입력 요구사항:** 폼 엘리먼트는 1줄 텍스트형 인풋 `[title]` 영역과 장문형 개행이 지원되는 텍스트에리어 `[content]` 창으로 유기 분할 배분됩니다.
- **트리거 동작:** 마우스 클릭 액션 외에도 단독 `Enter` 신호 인지 시 폼 자동 서브밋을 유발합니다.
- **예외적 텍스트 이스케이프:** `Shift + Enter` 제어 명령 동시 인입 시에는 폼 서브밋 프로세스를 의도적으로 우회 하며, 콘텐츠 에리어어 내부에 개행 정렬 바인딩을 온전히 실행합니다.
- **공백 방어 로직:** `title.trim() === ''` 혹은 `content.trim() === ''` 조건 참 발견 시 무조건 네트워킹을 정지시키고 인라인 방어 시스템을 전개합니다.

### 3.2 메모 조회 (Read)

- **마운트 페칭 시점:** 루트 뷰 컴포넌트가 최초 브라우저 트리에 등록되는 라이프사이클 초기 국면에 `useEffect` 를 활용하여 `GET /todos` 엔드포인트를 트리거합니다.
- **화면 상태 동기화:** 데이터 추가, 상태 변경, 영구 삭제 명령의 REST API 트랜잭션 수신 완료 직후 무조건 조회 레이 어를 재호출하여 로컬 메모리와 원격지 데이터 정합성을 항상 유지시킵니다.

### 3.3 메모 수정 및 중요도 토글 (Update)

- **컨트롤 포인트:** 렌더링된 카드 컴포넌트 내부의 중요 상태 지시 별표 아이콘 클릭 명령 접수 시, 대상 ID 식별자를 파라미터로 포함하는 `PATCH /todos/:id` 트랜잭션을 전송합니다.
- **시각 스타일 변화 연동:** 상태 수정 완료에 따라 카드 경계선 명도 가시성이 변경되며, 강조 배경 색채 조합 및 고정 특수 상태 태그 배지가 최상단에 실시간 인라인 재조정됩니다.

### 3.4 메모 삭제 (Delete)

- **트랜잭션 안전장치 보완:** 이벤트 핸들러 단계에서 즉시 물리 명령을 송신하지 않으며 브라우저 모달 대화창 `window.confirm()` 단계를 강제 삽입합니다.
- **의사결정 분기 제어:** 명시적 무효화 조작 수신 시 비동기 요청 호출부를 무조건 파괴하고 내부 동작 원형을 보존하여 오작동 데이터 유실 사고를 사전에 차단합니다.

### 3.5 검색 및 필터링 (Search & Filter)

#### 3.5.1 실시간 키워드 검색 및 텍스트 하이라이팅

성능 친화적 탐색 환경 조성을 위해 `useMemo` 를 통해 소스 스냅샷 가공 처리를 지탱합니다. 원본 제목 및 내용 검색 대상 텍스트 문자열 내에 입력 쿼리 패턴 포함 여부를 `includes` 논리식으로 스캐닝합니다. 일치하는 문자 블록에 직면하면 해당 범위를 정규식 및 분할 함수로 분리하며, 분리된 세부 요소에 고유 스타일 속성이 반영된 `<span>` 래퍼 태그를 인라인 주입 방식으로 동적 주입합니다.

#### 3.5.2 애니메이션 확장형 검색 제어 바 UI

기본 대기 상태에서는 최소 공간만을 점유하는 단순 돋보기 컴포넌트로 기능하다가, 사용자 활성화 조작 감지 시 내부 너비 변동 스타일 클래스인 `transition-all duration-300` 속성에 힘입어 수평 방향으로 부드럽게 확장 전개됩니다. 또한, 검색 타겟 범위의 외부 좌표 클릭 이벤트를 역추적 감지하는 영역 외 클릭 감지(Click Outside) 처리를 `useRef` 객체 및 문서 마우스 다운 이벤트를 기반으로 구축해 유연한 UI 접힘 및 초기화 동작을 실현했습니다.

#### 3.5.3 복합 다중 필터 전개 규칙

'중요 메모만 보기' 체크박스 선택 시, 활성화 플래그 조건과 검색어 패턴 매칭 상태 조건을 동시에 만족해야 하는 엄격한 `AND` 연산 논리 구조로 동작하여 방대한 텍스트 레코드 컬렉션 상황에서도 고속 다차원 필터링 정확도를 산출합니다.

## 4. 비기능적 요구사항

### 4.1 성능 최적화 (Performance Optimization)

대규모 컴포넌트 트리 렌더링 환경에서 유발되는 오버헤드를 제어하기 위해 본 프로젝트는 가상 DOM 재연산 비용 절감 기법을 체계화했습니다.

#### 4.1.1 `React.memo` — 컴포넌트 재렌더링 차단 고도화

목록에 표현되는 단위 요소 뷰 레이아웃 `MemoCard` 를 고차 컴포넌트(HOC)인 `React.memo` 구조로 선언했습니다. 이를 바탕으로 상위 부모 컴포넌트인 `App` 레벨의 전역적 데이터 상태 갱신 현상이 반복 발생되더라도 개별 메모 카드가 소유하는 독립 Props 참조 속성값의 동질성이 유지되는 한, 하위 컴포넌트 트리의 재연산 작업을 완전히 우회 패싱 처리하여 브라우저 가상 DOM 갱신 비용을 최소화합니다.

#### 4.1.2 `useCallback` — 힙 메모리 참조 주소 유지 관리

하위 컴포넌트로 주입되는 중요 제어 이벤트 콜백 핸들러 함수(예: `handleToggleImportant`, `handleDeleteMemo` 등) 정의 체계 전체를 `useCallback` 구조체 내부에 고정 격리시켰습니다. 컴포넌트 재생성 사이클마다 매번 불필요하게 힙 영역 내에서 참조 주소 바인딩 정보가 재생성되어 발생하는 자원 낭비를 완벽히 억제하며, 고유 참조 무결성을 보장하여 `React.memo`의 방어 장치가 깨짐 없이 안전하게 유지되도록 지원합니다.

#### 4.1.3 `useMemo` — 필터링 연산 결과 값 캐싱

사용자 글자 타이핑 인터랙션 동작 시 실시간 매칭을 처리하는 다중 필터 계산 로직은 배열 전체 요소를 전수 검사하므로 상대적으로 연산 자원 소모량이 높습니다. 이 로직 영역을 `useMemo` 디렉티브 내부로 엄격히 제한하여 데이터 배열 소스 원형 자체의 변경이나 명시적 필터 플래그 변동이 감지되지 않는 사소한 리렌더링 상황(예: 다른 인풋 포커싱 변동)에서는 연산 엔진 구동을 완전히 억제하고 캐싱되어 유지 중이던 직전 결과 객체 구조를 반환하여 연산 효율을 높였습니다.

### 4.1.4 useRef — 사이드 이펙트 배제 수단 활용

DOM 좌표의 내부 위치 검증 추적이나 입력 폼 요소의 강제 오토 포커싱 타겟 지정을 설정하는 영역에 `useRef` 참조 객체 풀을 할당했습니다. 렌더링 생명주기를 강제 제어 및 유발시키지 않고도 내부 메모리 상태 보존 값을 안전하게 핸들링할 수 있어 UI 조작 편의성과 최적화 정밀도를 크게 끌어올렸습니다.

## 5. UI/UX 요구사항

### 5.1 레이아웃 구조 설계 원칙

전반적인 웹 뷰 시각 인터페이스 아키텍처는 사용자 가독 편의성을 높이기 위해 **수평 방향 완전 가운데 정렬 시스템** 및 **싱글 컬럼 다운 방향 리스트 뷰 구조**를 엄격 준수합니다.

- **단계 1 [상단 헤더부]:** 서비스 대제목 타이틀 배치 및 하단 가로선 수평 레이아웃 경계 구획 정의
- **단계 2 [메모 입력 폼]:** 제목/내용 필드 영역 분배 및 우측 끝단 정렬 기반 파란색 액션 버튼 매핑
- **단계 3 [조건 필터 바]:** 잔여 개수 지표 표기, 애니메이션 슬라이더 확장형 🔍 버튼 및 중요 필터 래퍼 정렬 배치
- **단계 4 [카드 리스트 에어리어]:** 수직형 레이아웃 정형 배열 연동 및 데이터 상태별 조건부 배경색 스펙 전개

### 5.2 디자인 에셋 및 시각 스타일 규칙 (App.jsx 구현물 전용 컬러 가이드)

| 요소 구분    | UI 스타일 속성 및 구현 명세                       | Tailwind CSS 클래스 코드 예시                                |
|----------|-----------------------------------------|-------------------------------------------------------|
| 메모 추가 버튼 | Primary 컬러 (선명한 파란색 단색 채움 스타일)          | <code>bg-blue-600 hover:bg-blue-700 text-white</code> |
| 일반 메모 카드 | 기본 백그라운드 흰색 및 모던 얇은 회색 테두리 조합           | <code>bg-white border-gray-200 shadow-sm</code>       |
| 중요 메모 카드 | Amber/Yellow 포인트 테두리선 및 파스텔조 연노란색 배경 강조 | <code>bg-yellow-50/20 border-yellow-300</code>        |
| 삭제 버튼    | Danger 컬러 (고채도 적색 솔리드 배치로 경고 강조)        | <code>bg-red-500 hover:bg-red-600 text-white</code>   |
| 검색 강조 영역 | 매칭 키워드 대상 연한 파란색 하이라이트 인라인 바인딩          | <code>bg-blue-100 text-blue-800 font-semibold</code>  |



## 6. 향후 확장 계획

### 6.1 영속성 데이터베이스 레이어 도입

현재 적용 중인 데이터 적재 계층은 브라우저 런타임 및 서버 Node 인스턴스 휘발성 메모리 버퍼 상태에 의존하므로 인프라 다운타임 발생 시 모든 레코드가 유실되는 아키텍처 한계점을 내포합니다. 이를 개선하기 위해 단기적으로는 Node 네이티브 로컬 파일 시스템인 `fs.writeFile` API 기반의 파일 입출력 동기화 데이터베이스 구조 체계를 완성할 예정이며, 최종 진화형태로 PostgreSQL 계열 RDBMS 아키텍처나 유연한 Document 기반 NoSQL인 MongoDB 아키텍처 레이어를 결합함으로써 개별 태그 정보 및 메타데이터 적재 영속성을 영구히 확보할 계획입니다.

### 6.2 보안 고도화 및 인증 프로토콜 연동

인증 계층이 부재하여 발생하는 무방비 데이터 노출 위협을 차단하기 위해 향후 프론트/백엔드 단일 통신구조 인터페이스 사이에 JWT(JSON Web Token) 및 웹 세션 암호화 보안 미들웨어 체계를 확립할 예정입니다. 또한 데이터 입력 트랜잭션 수신 시 발생 가능한 SQL Injection 및 Cross-Site Scripting(XSS) 취약성 침투 시도를 필터링하기 위해 백엔드 익스프레스 라우터 초입부에 `express-validator` 검증 모듈을 필수로 결합하여 실무급 보안 신뢰도를 충족시킬 예정입니다.

### 6.3 다차원 기능 고도화

- **인라인 에디팅 모드:** 이미 생성 적재가 끝난 대상 메모 카드의 바디 콘텐츠 영역을 별도의 모달 이동 없이 브라우저 현 위치에서 실시간 상태를 재변경하는 인라인 폼 스위칭 편집 레이어 모듈 설계
- **태그 카테고리화 인덱싱:** 다중 멀티 태깅 처리를 위한 칩 UI 컴포넌트 추가 및 태그 식별자 기준 고속 다차원 복합 소팅 필터링 파이프라인 개설
- **다크모드 지원 테마 시스템:** Tailwind CSS의 `dark:` 가상 디렉티브 접두사 가이드를 빌드 프로세스에 이식하여 시각 피로도를 경감시키는 고품질 다크 테마 시스템 배포